

Java Developer Roadmap

A structured learning roadmap from Java fundamentals to backend development, projects, interview preparation, and job readiness.

1. Java Fundamentals

- Java syntax, variables, data types, operators, type casting
- Input/output using Scanner and basic console programs
- Conditional statements: if, else-if, switch
- Loops: for, while, do-while, nested loops
- Methods, parameters, return values, method overloading
- Arrays, strings, StringBuilder, common string operations
- Basic problem solving and time/space complexity

2. Object-Oriented Programming

- Classes and objects
- Constructors and constructor overloading
- this and super keywords
- Inheritance
- Method overriding and runtime polymorphism
- Method overloading and compile-time polymorphism
- Encapsulation and access modifiers
- Abstraction using abstract classes and interfaces
- Static, final, nested classes, packages

3. Collections Framework

- Collection hierarchy and when to use each collection
- ArrayList and LinkedList
- HashSet, LinkedHashSet, TreeSet
- HashMap, LinkedHashMap, TreeMap
- Queue, Deque, PriorityQueue
- Iterator and enhanced for loop
- Comparable vs Comparator
- equals() and hashCode()

4. Exception Handling

- try, catch, finally
- throw and throws
- Checked vs unchecked exceptions
- Custom exceptions
- Best practices for handling exceptions

5. Java Advanced Concepts

- Generics
- Lambda expressions
- Functional interfaces
- Stream API
- Optional
- Date and Time API
- File handling and I/O
- Multithreading basics
- Thread, Runnable, synchronization, executors
- JVM, JDK, JRE and basic memory concepts

6. SQL and Database Basics

- Relational database concepts
- Tables, primary keys and foreign keys
- SELECT, INSERT, UPDATE, DELETE
- WHERE, ORDER BY, GROUP BY, HAVING
- Joins: INNER, LEFT, RIGHT
- Subqueries and aggregate functions
- Indexes and basic normalization
- MySQL practice with Java applications

7. JDBC

- Connecting Java applications to a database
- Connection, Statement and PreparedStatement
- ResultSet
- CRUD operations

- Transactions
- Why PreparedStatement is preferred for user input

8. Spring Framework

- Why Spring is used in backend development
- Dependency Injection and Inversion of Control
- Beans and component scanning
- Annotations such as @Component, @Service and @Repository
- Configuration and application properties

9. Spring Boot

- Spring Boot project structure
- REST APIs
- Controllers, services and repositories
- Request parameters, path variables and request bodies
- GET, POST, PUT and DELETE
- DTOs and validation
- Exception handling
- Spring Data JPA
- Entities, repositories and relationships
- Pagination and sorting
- Configuration using application.properties or application.yml

10. Security and Authentication

- Authentication vs authorization
- Spring Security basics
- Password hashing
- JWT authentication
- Authorization headers
- Role-based access control
- CORS and CSRF concepts

11. Data Structures and Algorithms

- Arrays and strings

- Two pointers and sliding window
- Hashing
- Stack and queue
- Linked list
- Binary search
- Trees and binary search trees
- Heap and priority queue
- Graphs
- Recursion and backtracking
- Dynamic programming basics
- Practice coding problems regularly

12. Git and GitHub

- git init, clone, add, commit and push
- Branches and merging
- Pull requests
- Resolving merge conflicts
- Writing a useful README
- Keeping API keys and secrets out of repositories

13. Projects

- Build a CRUD REST API with Spring Boot and MySQL
- Build an authentication system using Spring Security and JWT
- Build a complete backend project with database relationships
- Build an AI-integrated application using an LLM API
- For each project, document features, architecture, technologies and setup steps

14. Interview Preparation

- Core Java interview questions
- OOP concepts and real-world examples
- Collections and HashMap internals
- Exception handling
- Java 8+ features and Streams
- SQL queries and joins

- Spring and Spring Boot fundamentals
- REST API and HTTP concepts
- JWT and security basics
- DSA coding practice
- Project explanation and resume discussion

15. Suggested Learning Order

- Step 1: Java fundamentals
- Step 2: OOP
- Step 3: Collections and exception handling
- Step 4: Advanced Java and Streams
- Step 5: SQL and JDBC
- Step 6: Spring and Spring Boot
- Step 7: REST APIs, JPA and Security
- Step 8: DSA and coding practice in parallel
- Step 9: Build 2–3 solid projects
- Step 10: GitHub, resume and interview preparation

Project Practice Checklist

- Can I explain OOP with examples?
- Can I choose the right Java collection for a problem?
- Can I write SQL joins and CRUD queries?
- Can I create a Spring Boot REST API?
- Can I connect Spring Boot with MySQL using JPA?
- Can I implement JWT authentication?
- Can I solve common DSA patterns?
- Can I explain my projects clearly in an interview?